



RV College of Engineering®

Autonomous institution affiliated
to Visvesvaraya Technological
University, Belagavi)

Approved by AICTE, New Delhi,
Accredited by NAAC, Bengaluru.

Go, change the world

Machine Learning over Encrypted Medical Data

A Project Report

Submitted by,

Mishma Toppo

Sarvesh Shashidhar

Veeradhi Rohit

1RV17EC076

1RV17EC137

1RV17EC178

Under the guidance of

Dr. Shylashree N.

Associate Professor

Dept. of ECE

RV College of Engineering

In partial fulfillment of the requirements for the degree of
Bachelor of Engineering in
Electronics and Communication Engineering
2020-2021

RV College of Engineering[®], Bengaluru

(Autonomous institution affiliated to VTU, Belagavi)

Department of Electronics and Communication Engineering



CERTIFICATE

Certified that the minor project work titled ***Machine Learning over Encrypted Medical Data*** is carried out by **Mishma Toppo (1RV17EC076)**, **Sarvesh Shashidhar (1RV17EC137)** and **Veeradhi Rohit (1RV17EC178)** who are bonafide students of RV College of Engineering, Bengaluru, in partial fulfillment of the requirements for the degree of **Bachelor of Engineering in Electronics and Communication Engineering** of the Visvesvaraya Technological University, Belagavi during the year 2020-2021. It is certified that all corrections/suggestions indicated for the Internal Assessment have been incorporated in the minor project report deposited in the departmental library. The minor project report has been approved as it satisfies the academic requirements in respect of minor project work prescribed by the institution for the said degree.

Signature of Guide Signature of Head of the Department Signature of Principal

Dr. Shylashree N.

Dr. K. S. Geetha

Dr. K. N. Subramanya

External Viva

Name of Examiners

Signature with Date

1.

2.

DECLARATION

We, **Mishma Toppo** , **Sarvesh Shashidhar** and **Veeradhi Rohit** students of seventh semester B.E., Department of Electronics and Communication Engineering, RV College of Engineering, Bengaluru, hereby declare that the minor project titled '**Machine Learning over Encrypted Medical Data**' has been carried out by us and submitted in partial fulfilment for the award of degree of **Bachelor of Engineering in Electronics and Communication Engineering** during the year 2020-2021.

Further we declare that the content of the dissertation has not been submitted previously by anybody for the award of any degree or diploma to any other university.

We also declare that any Intellectual Property Rights generated out of this project carried out at RVCE will be the property of RV College of Engineering, Bengaluru and we will be one of the authors of the same.

Place: Bengaluru

Date: 18/01/2021

Name

Signature

1. Mishma Toppo(1RV17EC076)
2. Sarvesh Shashidhar(1RV17EC137)
3. Veeradhi Rohit(1RV17EC178)

ACKNOWLEDGEMENT

We are indebted to our guide, **Dr. Shylashree N.**, Associate Professor, RV College of Engineering . for the wholehearted support, suggestions and invaluable advice throughout our project work and also helped in the preparation of this thesis.

We also express our gratitude to our panel members **Dr. Shylashree N**, Associate Professor and **Dr. Abhay A. Deshpande**, Associate Professor, Department of Electronics and Communication Engineering for their valuable comments and suggestions during the phase evaluations.

Our sincere thanks to **Dr. K. S. Geetha**, Professor and Head, Department of Electronics and Communication Engineering, RVCE for the support and encouragement.

We express sincere gratitude to our beloved Principal, **Dr. K. N. Subramanya** for the appreciation towards this project work.

We thank all the teaching staff and technical staff of Electronics and Communication Engineering department, RVCE for their help.

Lastly, we take this opportunity to thank our family members and friends who provided all the backup support throughout the project work.

ABSTRACT

Increased digitisation of data, and more people making avail of digital data based services coupled with a reported increase in data security breaches world over brings into question the current encryption-decryption model implemented throughout all security systems implemented for data protection. A growing problem in particular is medical data breaches, exposing highly confidential data in gross violation of Patient Confidentiality and Privacy laws. Furthermore, this medical data is being recorded at an exponential rate, with digitalised healthcare becoming a norm and old records being digitised for ease of its handling and analysis.

Through the course of technical implementation, a framework is developed that allows medical establishments like hospitals, research centres and pharmaceutical firms to analyse medical data without being explicitly aware of what the data includes. Most of these analyses based operations are handed over to Third Party Services due to the sheer volume of data being handled. It allows Data Owners to encrypt the data and allows Third Party Services to work on the data without actually having to decrypt it. This can be realised by applying Homomorphic Encryption on the data and can be easily coded in High Level Programming Languages like Python 3.

The result of the implementation of the algorithm in code form is successful implementation Homomorphic Encryption on CSV data using the Diffie-Hellman-Merkel Key Exchange, practically showing that the theoretical time and observed time for the process are very similar and that the model has an error rate of $37.1 \times 10^{-12} \%$. However, at this current stage, the model is highly processor intensive and takes to train.

CONTENTS

Abstract	i
List of Figures	v
List of Tables	vi
Abbreviations	vii
List of Symbols	viii
1 An Introduction to Machine Learning over Encrypted Medical Data	1
1.1 Introduction	2
1.2 Problem statement	3
1.3 Motivation	3
1.4 Objectives	3
1.5 Literature Review	3
1.6 Brief Methodology of the Project	5
1.7 Assumptions made / Constraints of the Project	6
1.8 Organization of the report	6
2 Homomorphic Encryption: A Comprehensive Overview	7
2.1 The Mathematics of Homomorphic Encryption	8
2.2 Types of Homomorphic Encryption and Schemes Implemented	9
2.3 Basics of Machine Learning	11
2.4 Why Implement on Medical Data	12
2.5 Secure Distribution and Privacy Preservation	12
2.5.1 Secure Two-Party Protocol	13
2.5.2 Secure Multi-Party Protocol	14
3 Design of Privacy Preserving Framework	15
3.1 Overview of the Framework Design	16
3.2 Pre-analysis of the Input Data	17
3.2.1 Imputing NULL values	17

3.2.2	Encoding String Data	17
3.3	Design Methodology	18
4	Implementation of Privacy Preserving Framework	20
4.1	Implementation Entities	21
4.1.1	The Data Owners	22
4.1.2	The Serving Gateways	22
4.1.3	Third Party Cloud Based Storage	23
4.1.4	Third Party Cloud Based Analytical Tool	23
4.1.5	Patient	23
4.2	Flow of the Framework	24
4.3	Implementation	24
4.3.1	Importing the Required Libraries	25
4.3.2	Linear Regression Model	25
4.3.3	Generation of Key Pairs	26
4.3.4	Getting the Keys	27
4.3.5	Data Serialisation	28
4.3.6	Loading the Data	28
4.3.7	Result Computation	29
4.3.8	Serialising Data	29
4.3.9	Getting the Encrypted Answer	29
4.3.10	Diffie-Hellmann-Merkel Key Exchange Algorithm	30
4.4	Prediction using Privacy Preserving Framework	31
4.5	Training the ML Model	32
5	Results & Discussions	34
5.1	Execution Results	35
5.2	Comparative Study	36
5.2.1	Predictive Framework vs Regular Linear Regression Prediction	37
5.2.2	Training Framework vs Regular Linear Regression Training	37
6	Conclusion and Future Scope	39
6.1	Conclusion	40
6.2	Future Scope	40

6.3	Learning Outcomes of the Project	41
-----	--	----



LIST OF FIGURES

3.1	Framework Overview	16
3.2	First 10 rows of the Dataset	17
3.3	Dataset Post-Analysis	18
3.4	Detailed Timeline	18
3.5	Design Methodology	19
4.1	Linear Regression Model	25
4.2	Paillier Key Pair Generation	26
4.3	Getting the Keys	27
4.4	Data Serialisation	28
4.5	Load Data	28
4.6	Compute the Answer	29
4.7	Diffie-Hellmann-Merkel Key Exchange Algorithm	30
4.8	Prediction Framework	31
4.9	Training Framework	32
5.1	Output for Prediction Framework	36

LIST OF TABLES

5.1	Timing Analysis of Predictive Framework Functions	37
5.2	Timing Analysis of Training Framework Functions	38



ABBREVIATIONS

CSP Cloud Service Provider

DO Data Owner

FHE Fully Homomorphic Encryption

HE Homomorphic Encryption

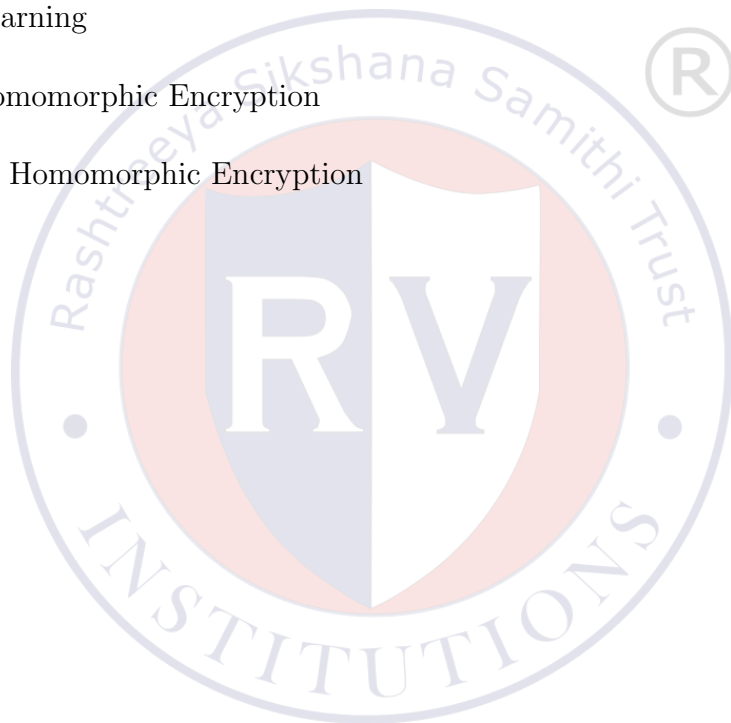
LHE Level Homomorphic Encryption

LR Linear Regression

ML Machine Learning

PHE Partial Homomorphic Encryption

SHE Somewhat Homomorphic Encryption



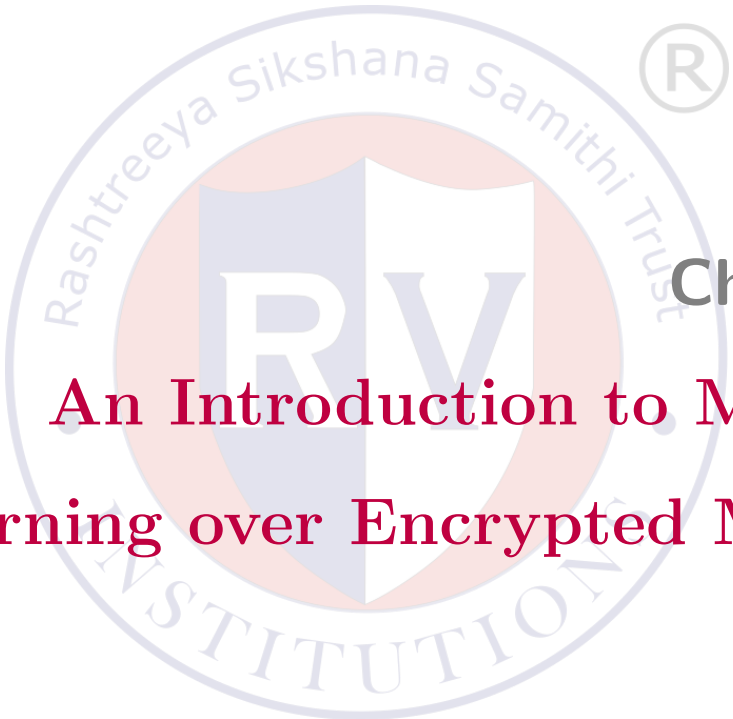
LIST OF SYMBOLS

$\mathcal{C}$ Cloud algebraic system

ϕ Decoding function used in Homomorphic Encryption

ϕ^{-1} Encoding function used in Homomorphic Encryption



The logo of RV Institute is a circular emblem. It features a central shield divided vertically into a light blue left half and a white right half, with the letters 'RV' in large, bold, white font across it. The shield is set against a red circular background. The outer ring of the emblem contains the text 'Rashtreeya Sikshana Samithi Trust' at the top and 'RV INSTITUTE' at the bottom in a light blue font. A registered trademark symbol (®) is located to the upper right of the emblem.

Chapter 1

An Introduction to Machine Learning over Encrypted Medical Data

CHAPTER 1

AN INTRODUCTION TO MACHINE LEARNING OVER ENCRYPTED MEDICAL DATA

Data, either stored or during transmission, the norm for cyber-security and data security is that it be encrypted. This proposed work will be discussing Machine Learning (ML) over encrypted data, the encryption technique being such that it offers the same security as other prevalent encryption techniques implemented while offering limited access to agreed upon data within the encryption.

1.1 Introduction

The arrival of big data has brought with it advances in processing technology, case in point, the healthcare industry resulting in an explosion of digitised healthcare data in the form of radiology reports, treatment planning, clinical decision support systems, diagnoses, point of care and data from personal health gadgets. Healthcare organisations have long implemented electronic medical record systems to manage and standardise the bulk amounts of data. Access to this data gives healthcare professionals an existing point of care to follow, researchers have an extensive sample of data to work with and through data analysis new data can be drawn from experiments. However, this cannot be done without infringing on patient - doctor confidentiality.

Let's assume that one jumps the patient - doctor confidentiality barrier by releasing only select pieces of data, one are then faced with advanced encryption standards that protect Personal Healthcare Information (PHI), Electronic Medical Records (EMR) and Clinical Data from Picture Archiving and Communication System (PACS). The study of this data would require the decryption of all this data to perform any form of computation on it. Decryption of this data is not only time cumbersome but also leaves this data vulnerable and is in complete violation of confidentiality laws.

Machine Learning offers a solution by virtue of its ability to work with large quantities of data. The training of a model that can work through encrypted data is not only presumable, but probable through the study of Homomorphic Encryption (HE), which retains both the encryption as well as the privacy of the data.

1.2 Problem statement

To create a sustainable framework for training analytical models and performing prediction tasks on encrypted data so that various data providers can collaborate without revealing any information about them or the data they are providing.

1.3 Motivation

Medical data theft and leakage is a growing problem worldwide, with a study indicating that in the last decade (2005 - 2019) around 250 million individuals were affected by data breaches [1], of which 158 million were in the last five years. In the last three years alone, medical data makes up 24% of all data breached, with the number of records exposed illegally or stolen in 2019 being 41.2 million (from 505/2013 medical data breaches) [2]. These numbers indicate a clear need for an improved data privacy and security model.

This area for progress, coupled with HE's capability to allow for selective data access, allowing collaborative research on preexisting medical data in its encrypted form allowing the training of ML models that can generate diagnoses for a patients presenting symptoms[3] and can detect patterns of a few diseases given their past records [4] is the main driver of this work.

1.4 Objectives

1. Manage and Analyse encrypted medical data in a CSV format to train a logistic regression model - implementing HE in a Client - Server - Receiver Model.
2. Model the two entities, sender and receiver using modular programming constructs.
3. Preserve the privacy of data owners and their data using this model.

1.5 Literature Review

With the rapid growth of data digitisation and with the introduction of big data analytics, a trend of large raw data transfer is being observed in all fields across the globe. The medical and healthcare industry is no exception to this. In fact, the data being sent over healthcare networks is usually Protected Health Information (PHI) which is highly confidential and it is highly risky to let this data leak out [1]. It not only causes harm to the Patient but can also cause a lot of damage to the healthcare provider and the Health Insurance Provider as well. A study performed [2] had revealed a whopping \$125

million dollar loss faced by the healthcare industry alone as a result of leaked PHI. This has raised a lot of concerns and a method for secure data transmission is also needed.

Another facet of handling medical data is the analysis that might be performed on it. With the large inflow of medical data, it is quite difficult for a single healthcare provider to go through all the data and analyse it in time for diagnosis. This data needs to be analysed using automated tools and computation algorithms. Luckily, the basic concepts of Machine Learning [3] play a significant role in this analysis process. For example, the use of clustering algorithms [5] like K – means clustering and Logistic Regression [6] can help analyse an MRI scan [4] and then identify the irregularities in the grey matter thus indicating the presence of a neuro-degenerative disease like Parkinson, Alzheimer's etc. Not only that, the concept of Recurrent Neural Network combined with HE can be used for [7] Secure Face Matching Applications. Recently, efficient HE algorithms for Medical Imaging [8] have also been developed and these algorithms have also been hosted on Cloud Based Service Platforms for extra portability.

Over the years, it has become a norm for the big healthcare organisations like Hospitals and Large Clinics to send the huge data volume to smaller Pharmacies for performing this Analysis. In this scenario, the merging of the two concepts of Data Security and Data Computation is seen which results in a new research domain termed as Secure Computations [9]. This field of study mainly deals with algorithms and protocols that can be applied to data for both – performing computation while maintaining the required level of security. It aims to expand [10] on topics like data auditing and accounting which act as the base for most of the secure computations in today's data driven world.

At this point, the question of how data can be kept secure throughout the entire computation and transfer process keeps popping up. Luckily, a familiar and clear answer [11] was available all this time – cryptography. This field concerns itself with creating algorithms to code a data input so that only select parties with the decoding algorithm can view the data fully. Numerous algorithms have been developed in the said field [12] and most of them have been implemented as a part of network security. Not only has the process of encryption made it difficult for unauthorised parties to tap into conversations and exploit the data being transferred, it has also created a space for [13] development of numerous information systems that can exploit the cryptographic concepts and create a secure data framework.

With the knowledge of cryptographic algorithms present, it is crucial to select the algorithm that best suits the requirements. [14] This algorithm must not only be secure and difficult to break into, but must also be computationally light and not processor intensive. Moreover, it must be implementable on the communication protocols that are being used in the framework. Keeping all these points in mind, Homomorphic Encryption schemes [15] were employed. These schemes are a simple yet efficient method to protect the data and also enable parties to work on the data without decrypting it. Basically, it provided the programmer with the power of keeping the data a black box throughout the computation process and thus maintaining a high level of data security.

The concept of homomorphic encryption [4] [16] has been applied to Medical Data in the upcoming chapters and has also been used in conjunction with [17] Machine Learning Algorithms. Even in the medical field, a [18] predictive analysis is widely used as it can help predict any future ailments that might torment the patient given past data. Now, this analysis can be performed on secure [19] and encrypted data [20] making it a win – win situation.

Finally, as the data is not local and is often sent to various parties [21] for analysis, this secure data protocol and encryption scheme must be applied to not only the Data Owners but also to the sub-ordinate parties. This calls for a framework that emulates a collaborative association [22] between the various parties involved in the data analysis without risking the data fields and values. Creation of such a framework has been the goal for many researchers and various solutions have been put forth.

We can glean, from all this information that it is not a new concept, HE, and there have been many practical implementations of it in a variety of data security fields, with the medical field being a key area of interest. This is due to the untapped potential for growth and development in the machine learnt medical diagnosis field and the number of diseases and disorders that can be classified by the study and analysis of its patterns.

1.6 Brief Methodology of the Project

The literature survey highlights the use of HE as a means of achieving our set objectives in the following series of steps:

1. Implement Partial Homomorphic Encryption (PHE) on a programming platform.
2. Develop and train a Linear Regression Model for regression problems and a Logistic

Regression Model for classification problems.

3. Assign roles to each member of the team as

- Member 1 acts as the Data Owner.
- Member 2 acts as the Cloud Service Provider (CSP) (Server).
- Member 3 acts as the Data Operator.

4. Simulate the Black Box Data Computation process and rectify errors arising during the process.

1.7 Assumptions made / Constraints of the Project

Due to the nature of the data involved and the algorithm used, the set-up and execution was done via software and there is no hardware component involved.

The second party in the Client-Server-Receiver Model, although traditionally a third-party CSP, was substituted for a modular programmed entity to reflect this behaviour.

1.8 Organization of the report

This report is organized as follows. Write the discussions in each chapter. A sample is as follows.

- Chapter 2 discusses the fundamentals of HE and details its operation.
- Chapter 3 details the design process of the model, highlighting the network structure taken into account for a real world scenario.
- Chapter 4 describes the model and elaborates on its real word implementation.
- Chapter 5 presents the results obtained over the course of the project.
- Chapter 6 is the conclusion of the report, and further discusses if there is any future scope for the project.



Chapter 2

Homomorphic Encryption: A Comprehensive Overview

CHAPTER 2

HOMOMORPHIC ENCRYPTION: A COMPREHENSIVE OVERVIEW

HE, originally introduced as a "special privacy homomorphism" in Rivest and Adelman's 1978 paper [9] is a special encryption function that works on a data set in a manner that makes redundant the decryption of encrypted data. It was offered as a solution to a problem where a loan company makes use of a commercial time shared server, but data access meant decrypting the data, and at each instance compromising it. HE would allow access to the data through the use of boolean or mathematical functions.

2.1 The Mathematics of Homomorphic Encryption

HE is a crypto-system, and its main intended function as such is security of the data it is employed upon and should maintain the basic requirements of any cryptographic algorithm - its ability to work over a diverse range of data types, an ease of revocability of compromised data sets, the security of the data it encrypts, fast operational times of both the encryption and the data access mathematical operations, and its robustness in preventing the reverse engineering of the original data from an encrypted sample if made available [10] [11].

This security establishment is done via the use of keys, a unique alphanumeric entity distributed among the concerned parties as

- Public Keys, known by all parties regardless of involvement in data exchange.
- Private Keys, known only by the data owners.

making HE a public-key cryptography system. Decryption, or in this scenario, access to the data is allowed only through the use of the private key, with the encryption process allowed by anyone through the public key [12].

Adelman *et al* [9] lay out the algebraic system of HE as a set S consisting of arithmetic operations f , predicate p and distinguished constants s related to the algebraic system of the user U and the algebraic system C used by the computer system (cloud) as

$$S = (S; f_1, f_2, \dots; p_1, p_2, \dots; s_1, s_2, \dots) \quad (2.1)$$

$$U = (S; f_1, \dots, f_k; p_1, \dots, p_k; s_1, \dots, s_m) \quad (2.2)$$

$$C = (S'; f'_1, \dots, f'_k; p'_1, \dots, p'_k; s'_1, \dots, s'_m) \quad (2.3)$$

In a regular cryptographic systems have the decoding function ϕ and the encoding function ϕ^{-1} as

$$\phi = S' \rightarrow S \quad (2.4)$$

$$\phi^{-1} = S \rightarrow S' \quad (2.5)$$

But under HE, for the operation without decryption, ϕ must be a homomorphism of C on U . Thus homomorphism translates to three key points, for every *ith* of the

- arithmetic operation f applied on the data set, it equals the arithmetic operation applied on the decoding function applied on the data set.
- predicate operation p applied on the data set, it equals the predicate operation applied on the decoding function applied on the data set.
- distinguished constant operation s , it equals the decoding function applied on the the distinguished constant.

This essentially means that ϕ carries into C , and as it is a homomorphism, the system can access the encrypted form without having to decrypt.

From this, the requirements of the HE system and ϕ^{-1} and ϕ functions are ascertained to be easy to compute, the arithmetic function and predicate should be efficiently computable, knowledge of the plaintext or cipher text alone should not be sufficient to reveal the data. This satisfies the requirements of any key based crypto-system [13].

2.2 Types of Homomorphic Encryption and Schemes Implemented

Over the course of time, HE has subdivided into a variety of types, each varying in the arithmetic computations involved in the data retrieval process. This is because real

word applications require only some of the functionality, and designing to perform only the required operations decreases the time taken to perform homomorphic operations and makes the schemes employed more compact as well [15]. An example of this is in the medical, advertising and financial domains, where it is enough to perform only a multiplication or addition operation on the ciphertext.

The main types used practically are

- PHE which supports a single arithmetic encryption operation either addition or multiplication.
- Somewhat Homomorphic Encryption (SHE) which supports a limited number of both additions and multiplications.
- Fully Homomorphic Encryption (FHE) which supports unlimited additions and multiplications directly in the encrypted domain.
- Level Homomorphic Encryption (LHE) which supports any arithmetic operation, but at a fixed parameter length.

Each types serves as an umbrella term for varied schemes, available either publicly or are proprietary. These schemes are designed around a select choice of parameters, as illustrated by Lauter *et, al* [15], where the scheme employed is built on a sub-main type of HE, and around the operations involved in the ML training process and how the data once retrieved is to be treated and the degree of correctness required.

The parameters training the model too are based around these design points, to leverage the security aspect of HE, with respect to the key length and its ease of operability to security offered. Most common parameters considered are

- the degree of the polynomial used for evaluation
- the maximum of the coefficients in the polynomial
- the size of integers handled during the computation process
- the desired security level by the encryption scheme
- the allowable error as a standard deviation
- the work size of the key used

These design variations and capability to build upon make HE inherently malleable in nature.

2.3 Basics of Machine Learning

The main analysis method used in the framework is the ML Linear Regression (LR) model. ML as a concept deals with the fact that a machine can be trained and coded to perform a certain task without human intervention. Basically, a machine is made to *learn* how to perform the said task. This field has been the epicentre of problem solving research for a good part of decade and various ML algorithms have been developed for this very purpose. In general, these algorithms can be divided into two types

- **Regression** - These are the models that focus on learning from past data and then predicting the possible output for new unseen data inputs
- **Classification** - These are the models that focus on learning from past data and then trying to classify the incoming unseen data into the respective groups.

For simplicity and effectiveness, the proposed work deals with an LR model. As the name suggests, the LR model belongs to the regression models class. It focuses on using past data to get a best fit line. This is an algorithm that models the relationship between two variables by fitting a linear equation to observed data. One variable is considered to be an explanatory variable and the other is considered to be a dependent variable.

A linear regression line has an equation of the form $Y = a + bX$, where X is the explanatory variable and Y is the dependent variable. Before fitting a linear model to observed data, it needs to be determined whether or not there is a relationship between the variables of interest. The most common method for fitting a regression line is the method of least-squares.

This method calculates the best-fitting line for the observed data by minimising the sum of the squares of the vertical deviations from each data point to the line. Mathematically a and b are calculated as follows -

$$b = \frac{n \sum xy - (\sum x)(\sum y)}{n \sum x^2 - (\sum x)^2} \quad a = \frac{n \sum y - b \sum x}{n} \quad (2.6)$$

Python 3 is a high level language that has previously defined modules for Machine

Learning models and the module for LR is directly used in the framework.

2.4 Why Implement on Medical Data

ML serves as an excellent tool in the classifying of patterns, which is exactly why it is such an important tool in the medical data analysis field - with its capabilities to be trained to work on a ginormous amount of data. Further more, increasing amounts of medical data are being digitised for ease of handling and operability, creating a massive back-catalogue of data to research upon. The possibility of working on this to develop models based on ailment patterns can aid in the diagnosis of the presented disease [4]. Furthermore, ML can also aid in early diagnosis and help in the development of new therapies for the treatment of diagnosed diseases. The advantage ML has over traditional computing is in its ability to *learn* from the changes in patterns and predict based off these changes.

However, there is a constraint. These large amounts of data are to be easily accessible for the computer to learn from them, this access is limited by Patient Confidentiality, and require decryption first. Cloud platforms, like Microsoft Azure are developing models for encryption using HE to make predictions using encrypted data while having the contents of the data still be obscure from the operating party. A fully realised example of this is Bos *et, al*'s [20] proof of concept wherein they return the probability of a cardiovascular disease determined by ML on data using HE. This can be achieved by choosing one of the many HE schemes and fine-tune the parameters of the data accessed to guarantee both the security of data and the correctness of the prediction made off it.

2.5 Secure Distribution and Privacy Preservation

When one talks about handling such large quantities of data, more often than not, it is not a one-party application. Multiple parties host or supply the data worked upon, and multiple parties could be allowed access to work upon designated aspects of data. Usability of this data is thus dependent on the ability to compute on desired attributes in a multi-party scenario. This is illustrated in Zhan *et, al*'s [22] paper on collaborative association rule mining, where using data mining over encrypted data, they develop multi-part protocols, namely - the secure two party protocol and the secure multi-party protocol. These are designed based on the the secure multi-party computation model as introduced by Yao [14], which while allowing multiple access by giving each user a share

to access, is impractical when working with large datasets.

Borrowing from Zhan [22] the theory behind involved in designing these protocols, while generalising that ML over encrypted data, trained in a manner to look for defined pieces of information is a variant on data mining, protocols defined in this paper can be used as a basis to define the interaction of the model to be secure distributed - *i.e.*, to obtain desired outputs for multiple parties. It makes use of the rule of associativity of HE functions

$$e(a_1) \times e(a_2) = e(a_1 + a_2) \quad (2.7)$$

where e is the encryption function, a_1, a_2 are the data to be encrypted. By associativity

$$e(a_1 + a_2 + \dots + a_n) = e(a_1) \times e(a_2) \times \dots \times e(a_n) \quad (2.8)$$

where

$$e(a_i) \neq 0 \quad (2.9)$$

The association rule mining determines the support offered by a dataset as

$$\frac{c.count}{\text{total number of data records}} \quad (2.10)$$

2.5.1 Secure Two-Party Protocol

In this scenario, we consider two parties, with vectors A_1, A_2 with N elements each.

- The number of transactions are N
- A random $X > N$ is selected
- Party 1 generates a HE scheme key pair (d, e) , where $e(.)$ is encryption and $d(.)$ is decryption. Encryption performed on a set of random integers $R_1, R_2, \dots, R_N, X$ and vector A_1 using e is sent to Party 2.
- Party 2 computes the a value E , the product of the encryption received and vector A_2 . Values of $E \neq 0$ are summed up and sent back to Party 1.
- $c.count = d(E) \bmod X$

2.5.2 Secure Multi-Party Protocol

In this scenario, each party P_i of $P_1, P_2, \dots P_n$ has vectors $A_1, A_2, \dots A_n$.

- The number of transactions are N
- A random $X > N$ is selected
- Each P_i jointly generates a HE scheme key pair (d, e) , where $e(.)$ is encryption and $d(.)$ is decryption. Encryption performed by each P_i on a set of random integers $R_1, R_2, \dots R_N$, X and vector A_i using e is sent to P_n .
- P_n computes the a value E , the product of the encryption received and vector A_i .
- The E sequence is permuted and sent back to each of the sending parties.
- $c.count_i = d(E_i) \bmod X$
- $c.count = \Sigma c - count_i$
- The verification of $c.count$ is done by having a random P_i encrypt it with a separate key pair (d_i, e_i) and having the recipient decrypt it.

Due to the multiple operations involved in the multi-party setup, it is more computationally complex than the two-party setup. Establishing and understanding the requirements of the HE, the sort of inter-system communication it works with and the algorithms it works on allows the design and modelling of frameworks - both to train and predict an ML model that can achieve the objectives proposed. Once the theory and mathematical intricacies have been understood, the design of the proposed framework can be undertaken.



Chapter 3

Design of Privacy Preserving Framework

CHAPTER 3

DESIGN OF PRIVACY PRESERVING FRAMEWORK

The prime objective of the technical undertaking is to develop a sustainable framework using PHE and LR to maintain data privacy. The design of such a framework involves the modelling of various entities and their connections to each other. Following a brief insight into the design procedure and the flow of the framework design.

3.1 Overview of the Framework Design

The design of the framework models five entities in total -

1. The Data Owner (DO)
2. The Serving Gateways
3. The Cloud Storage
4. The Cloud Based Analytical Tool
5. The Patient

This has been represented in Figure 3.1 The DO encrypt the data and then send it to Cloud based tool for analysis. The encrypted answer is then sent back to the DO and the Patient for viewing.

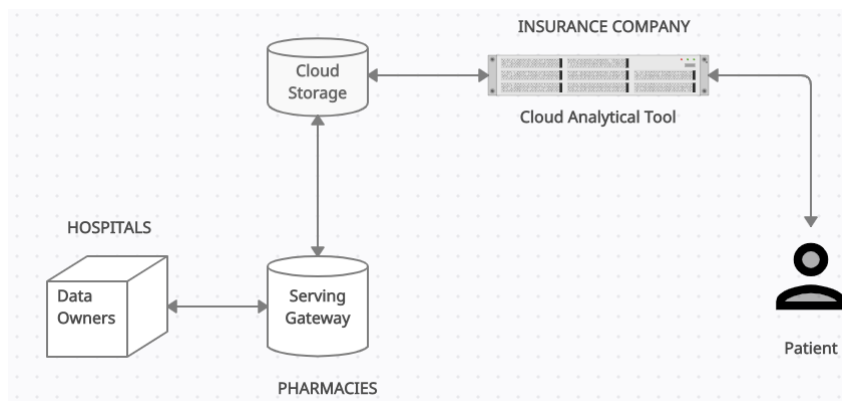


Figure 3.1: Framework Overview

Figure 3.1 illustrates the various entities and their communication pathways.

3.2 Pre-analysis of the Input Data

For the scope of this model, the data used is a set of Medical Insurance charges dependent on various factors associated with the patient. This data is stored in a CSV format and is structured for easier analysis and handling.

	A	B	C	D	E	F	G
1	age	sex	bmi	children	smoker	region	charges
2	19	female	27.9	0	yes	southwest	16884.924
3	18	male	33.77	1	no	southeast	1725.5523
4	28	male	33	3	no	southeast	4449.462
5	33	male	22.705	0	no	northwest	21984.47061
6	32	male	28.88	0	no	northwest	3866.8552
7	31	female	25.74	0	no	southeast	3756.6216
8	46	female	33.44	1	no	southeast	8240.5896
9	37	female	27.74	3	no	northwest	7281.5056
10	37	male	29.83	2	no	northeast	6406.4107

Figure 3.2: First 10 rows of the Dataset

However, it is clear from Figure 3.2 that the data is stored as floating values and strings. For a basic LR model, the data needs to be in floating point values and therefore introduces a need for data pre-analysis.

3.2.1 Imputing NULL values

The first step in the data pre-analysis is to impute NULL values in the Dataset. The presence of NULL values can cause disruption in the analysis and result in incoherent answers. To prevent that, the NULL fields were imputed with the *mean* of the respective column values. This can be done by using the *fillna* function that is present in Python 3 sklearn module.

3.2.2 Encoding String Data

After the NULL fields have been imputed, the focus is shifted on converting the string data in the dataset to numeric data. This is done by a process called *encoding*. The encoding process takes in the total number of values in the dataset and then assigns a single numeric value to each possible string data. For example, we have two values of the *Sex* field in the Dataset - Male and Female. So, the code shall encode *Male* = 0 and *Female* = 1 or vice-versa. The same process is then applied to the rest of the string data columns.

For this implementation, the encoding of string data is enough to complete data pre-analysis and this data (as shown in Figure 3.3) can now be fed to the LR.

	age	sex	bmi	children	smoker	region	charges
0	19	0	27.900	0	1	3	16884.92400
1	18	1	33.770	1	0	2	1725.55230
2	28	1	33.000	3	0	2	4449.46200
3	33	1	22.705	0	0	1	21984.47061
4	32	1	28.880	0	0	1	3866.85520

Figure 3.3: Dataset Post-Analysis

3.3 Design Methodology

The Design Methodology is as seen as in Figure 3.5 was developed by keeping the Objectives and the Timeline as mentioned in Figure 3.4.

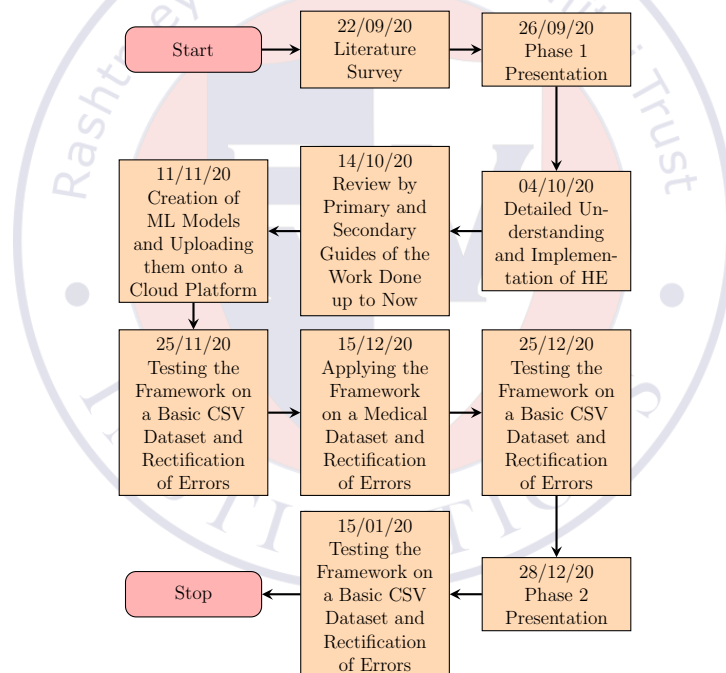


Figure 3.4: Detailed Timeline

The design methodology is given below -

1. The first task is to divide the Objectives into Pre-Design Goals. These goals are created to achieve an innovative solution to the Problem Statement at hand and to achieve the Objectives set at the beginning.
2. After that, the feasibility of these Pre-Design goals are weighed and then few innovative ideas are selected for implementation. For example, the innovative idea

chosen for implementation was PHE

3. Next step involves the implementation of the innovative part alone. This is to check if the idea is implementable and if there are any errors to be rectified when the innovation is implemented.
4. Next step involved the creation and implementation of the Analytical Model which is the LR model in our case.
5. Once these tasks are done, the next step is *Integration*. The PHE and the LR models were integrated into a single framework. This step marks the end of design and implementation.
6. The next part is the *Testing* part. The framework was first tested for basic CSV data. This was to isolate any errors and rectify them.
7. Once the framework was simulated for CSV data and error-free output was observed, the dataset was replaced by a bulkier medical data file.
8. With the observation of the error-free output for the Medical Data file concludes the proposed work and marks the end of the current scope.

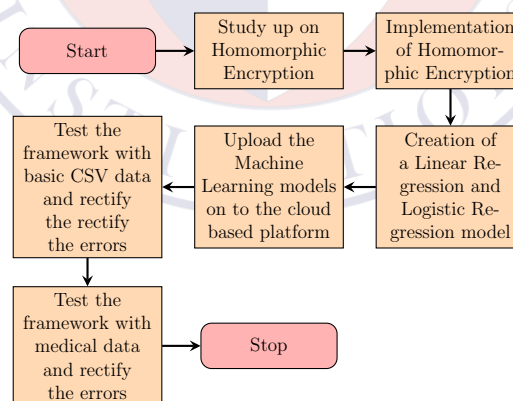


Figure 3.5: Design Methodology

Post theoretical design and implementation planning, the proposed work is moved into the practical implementation phase. In this, learnings from the theoretical observation and algorithm research is translated into high-level language code to execute for results. The further work will present a detailed implementation of the proposed model.



Chapter 4

- **Implementation of Privacy Preserving Framework**

CHAPTER 4

IMPLEMENTATION OF PRIVACY PRESERVING FRAMEWORK

The Privacy Preserving Framework was implemented using basic Python 3 constructs and encryption schemes. As mentioned previously, Jupyter notebook was used to model, code and realise the flow of the framework. The encryption module used was the **Partial Homomorphic Encryption (PHE)** library that is in-built with Python. This chapter expands upon the details of the framework implementation.

4.1 Implementation Entities

The implementation of the Privacy Preserving Framework involved modelling of five entities -

1. **The Data Owners** - These are the entities that provide the data for analysis. They are responsible for the encryption and de-cryption of the data using PHE. In our real - life example, the Hospitals act as the DO
2. **The Serving Gateways** - These are the entities that act as the servers in the framework. They feed the encrypted data from the DO to the third party Cloud Storage and vice-versa. In our real - life example, the Pharmacies act as the Serving Gateways or Servers
3. **Third Party Cloud Based Storage** - This is the Cloud Based Storage that stores the data coming from the Serving Gateways so that it may be used by the Cloud Based Analytical Tool.
4. **Third Party Cloud Based Analytical Tool** - This is the actual Analytical Tool that is used for performing analysis on the incoming Encrypted Data. For our real-world application, this tool is usually with the Insurance Company that acts as the third party. Also, for simplicity, Linear Regression was used for analysis.
5. **Patient** - This is the final entity in the framework and is the entity that needs to be aware of the analysis results. The patient is the only entity apart from the DO

that posses the ability to decrypt and view the results. This feature ensures that there is no breach of Patient-Doctor Confidentiality.

Each of these entities have separate roles and thus use separate functions.

4.1.1 The Data Owners

The DO are the entities that have the original Patient data with them. This data is sensitive and therefore needs to be encrypted before sending it to the other entities. The data is encrypted using **Partial Homomorphic Encryption** which has been elaborated upon in the previous chapter. This encryption results in a long integer value that reveals no information about the original data whatsoever.

The original data is encrypted using the Public Key that is assigned to the DO. The real intricate beauty of PHE method lies in the fact that the Public Key can be used for encryption process but the data can be decrypted only by using the Private Key. This means that the other entities on the framework (who are aware of the Public Key assigned to the DO) can't decrypt the data using the Public Key. This makes PHE ideal for our application.

Once the data has been successfully encrypted, it is sent to the Serving Gateway which is the next node in the framework. The connection between the DO and the Serving Gateway is secured by Diffie-Hellmann-Merkel Key Exchange Algorithm. For the real world Medical Field Application, the Hospital with the Patient records and data acts as the DO.

4.1.2 The Serving Gateways

The Serving Gateways (Servers) are the nodes that connect the DO to the Third Party entities. The serving gateway doesn't perform any analysis or operation on the encrypted data which it has received from the DO, but ensures that the Third Party tools do not directly communicate with the DO. This gives the DO additional security against malicious entities.

The Serving Gateways provide a secure path for encrypted data to move from the DO to the Third Party Cloud Storage and vice-versa. This means that there are two data paths from and to the Serving Gateway which use the DHM Key Exchange Algorithm for security. In the medical application, the Pharmacies act as the secure gateways for the data to flow.

4.1.3 Third Party Cloud Based Storage

This is the first Third Party Entity that is encountered in the framework. It is used to store the incoming encrypted data from the Serving Gateway so that the Third Party Analytical Tools can access the said data for analysis.

4.1.4 Third Party Cloud Based Analytical Tool

This is the main analytical tool that performs the analysis. There are two kinds of operations that are performed on this tool -

- To use trained models present on the cloud for performing analysis on encrypted data
- To use the encrypted data for training the model hosted on the Cloud Platform

For the analysis part, the data remains encrypted throughout and therefore, is secure from any sort of malicious interference. While on the other hand, the training of the model being hosted on the cloud is done by first decrypting the data to the original form and using that to perform the training process.

This entity is a model of real-world Insurance Companies and therefore, by signing appropriate agreements between the Hospitals and the Insurance Companies, the training of the analytical model by using original data can be accomplished without major compromises.

As mentioned previously, the analytical tool used to perform said analysis is a Machine Learning Model that uses the Linear Regression method for predicting the required values for a near future scenario.

4.1.5 Patient

The final node in the framework is the Patient itself. The original data that is present with the DO has been collected with the help of the Patient and therefore, the analysed data is sent to the Patient as well. The data is however encrypted and the Patient is provided with the means to decrypt the data so as to view the analysed outputs. Since the data belongs to the Patient in the first place, there is no breaching of protocols or standards when the Patient is given full access of the analysed outputs.

4.2 Flow of the Framework

Once the entities have been modelled and their connections established, the flow of data under the correct operating conditions follows the given below flow. However, before proceeding with the flow of data in the framework, one must look at some pre-requisites as listed below -

- Each entity has a single Public Key n and two Private Keys p, q
- The data is being provided as a row (array) to the entities
- The data used for analysis is structured and is in the CSV format

Keeping these three points in mind, the flow of data can be easily understood -

1. The first step requires the DO to take the data to be analysed and then encrypt it using the PHE Scheme. The Public Key of the DO is used for the Encryption process.
2. Next, the encrypted data is sent to one of the Serving Gateways which securely sends the data to the Third Party Cloud Based Storage System.
3. After that, the Cloud Based Analytical Tool takes the encrypted data from the Cloud Based Storage System for analysis.
4. The Cloud Based Tool performs the necessary analysis and then generates an encrypted answer.
5. This encrypted answer is then sent back the same path to the DO and is also sent to the Patient
6. Finally, the patient and the DO decrypt the answer and get the final output.

4.3 Implementation

The code for the framework can be divided into an assortment of functions.

4.3.1 Importing the Required Libraries

The following libraries from the Python 3 Suite were imported -

- **Pandas** - For Dataframe handling and manipulation
- **Numpy** - For Data structure (like Array) handling
- **Time** - For calculating the time taken for various coding constructs
- **Sklearn** - For Machine Learning and Data Pre-processing
- **PHE** - The Partial Homomorphic Encryption Library
- **JSON** - To handle JSON files

4.3.2 Linear Regression Model

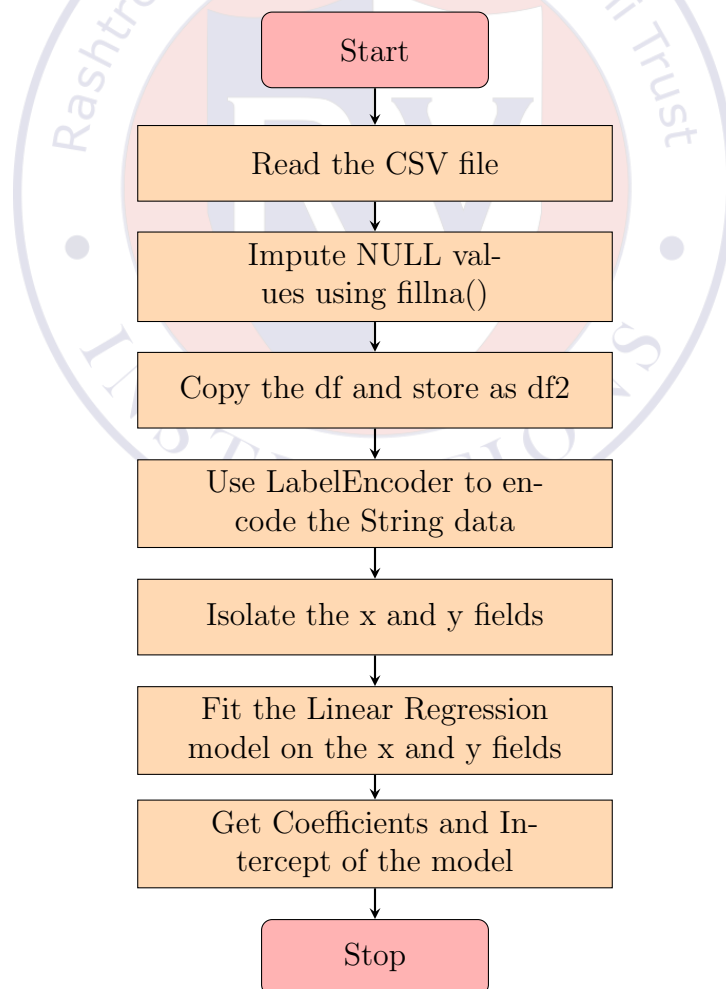


Figure 4.1: Linear Regression Model

This is the Linear Regression model used for analysis. This is exclusively used for the Prediction part of the framework which means that it is assumed that the Linear Regression model has already been trained and is present in the Cloud Based Analytical Tool. The general flow of a Linear Regression model is shown in Figure 4.1.

4.3.3 Generation of Key Pairs

The flow depicted in Figure 4.2 is responsible for generating a single Public Key and a pair of Private Keys for all the five entities involved. All the keys are stored in a dictionary datatype and then these dictionaries are dumped onto separate JSON files.

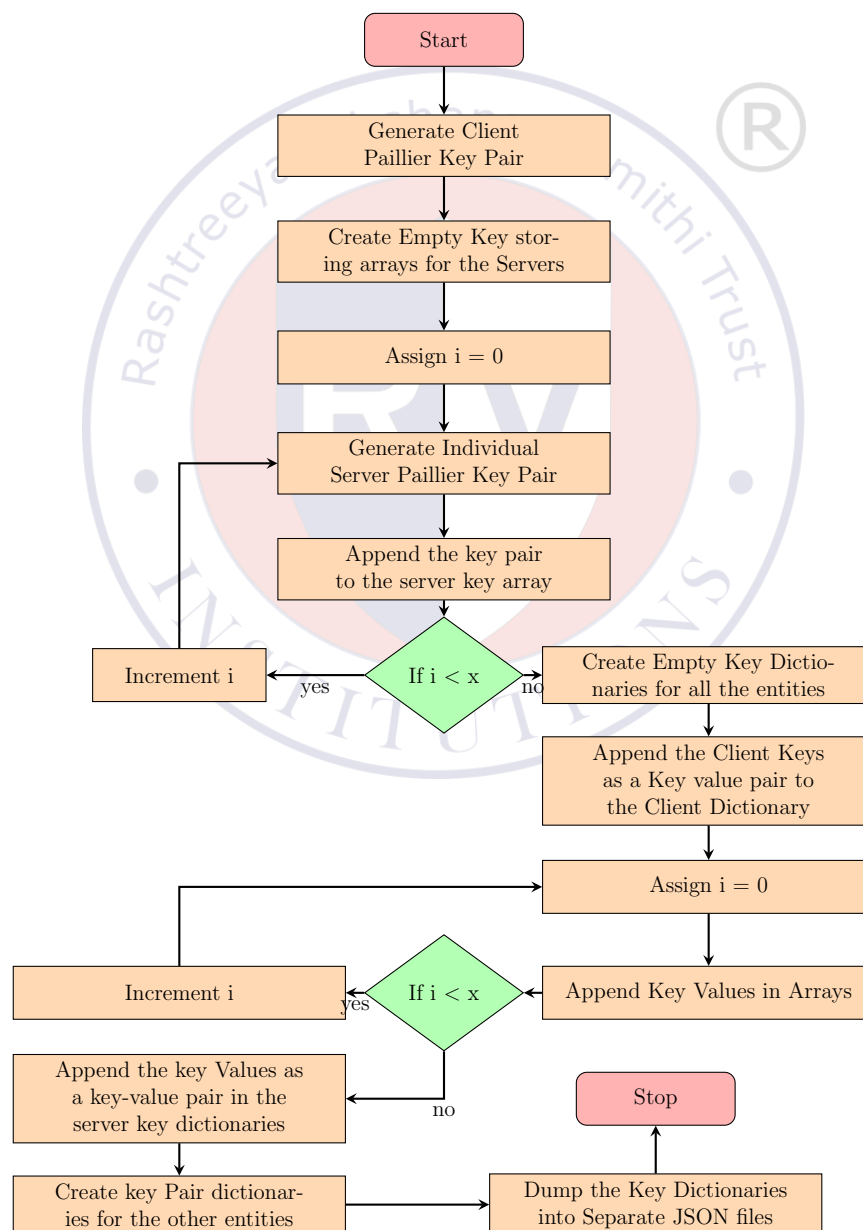


Figure 4.2: Paillier Key Pair Generation

4.3.4 Getting the Keys

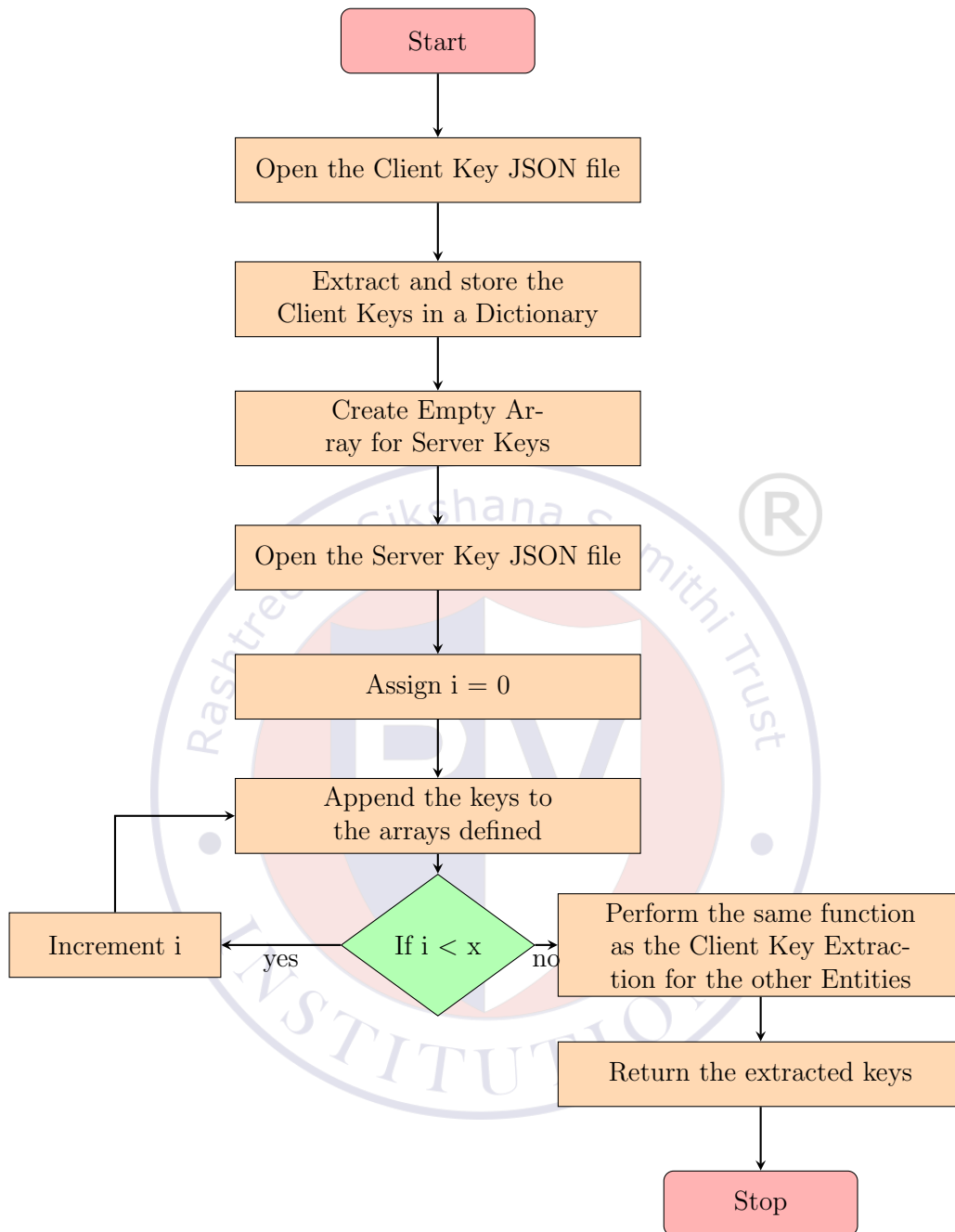


Figure 4.3: Getting the Keys

From Figure 4.3 it is evident that the PHE library functions are used to get the key pairs. This results in the keys being in a separate datatype and this makes them difficult to manipulate as regular numbers. Hence, this function takes in the keys in their native datatype and returns the keys as integers.

4.3.5 Data Serialisation

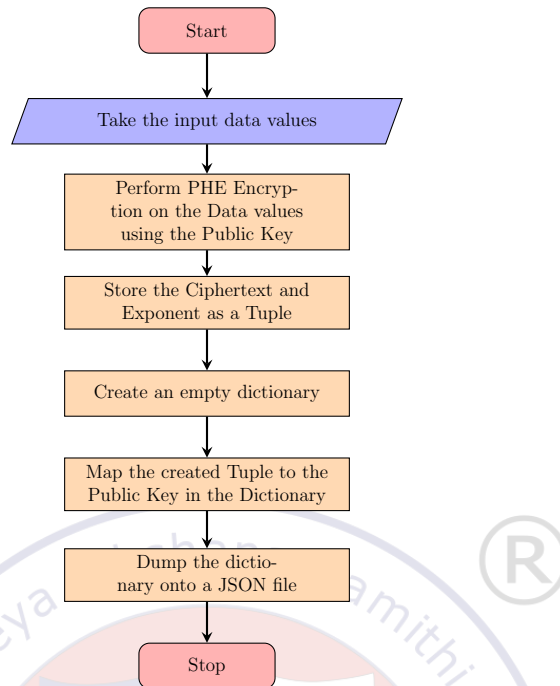


Figure 4.4: Data Serialisation

Figure 4.4 represents a flowchart which is responsible for the encryption of the data using the public key provided. It encrypts the data and then stores the ciphertext with the exponent in a dictionary. The dictionary is then converted to a JSON compatible format as a string.

4.3.6 Loading the Data

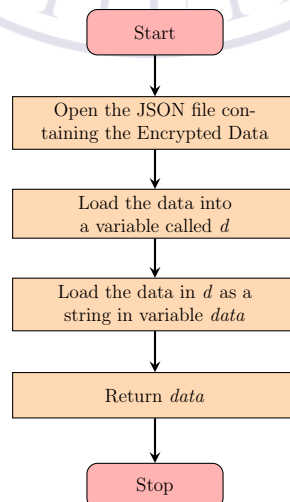


Figure 4.5: Load Data

Now that the data is present in a JSON file, the flow in Figure 4.5 is added to read the JSON file and then read the data present in it.

4.3.7 Result Computation

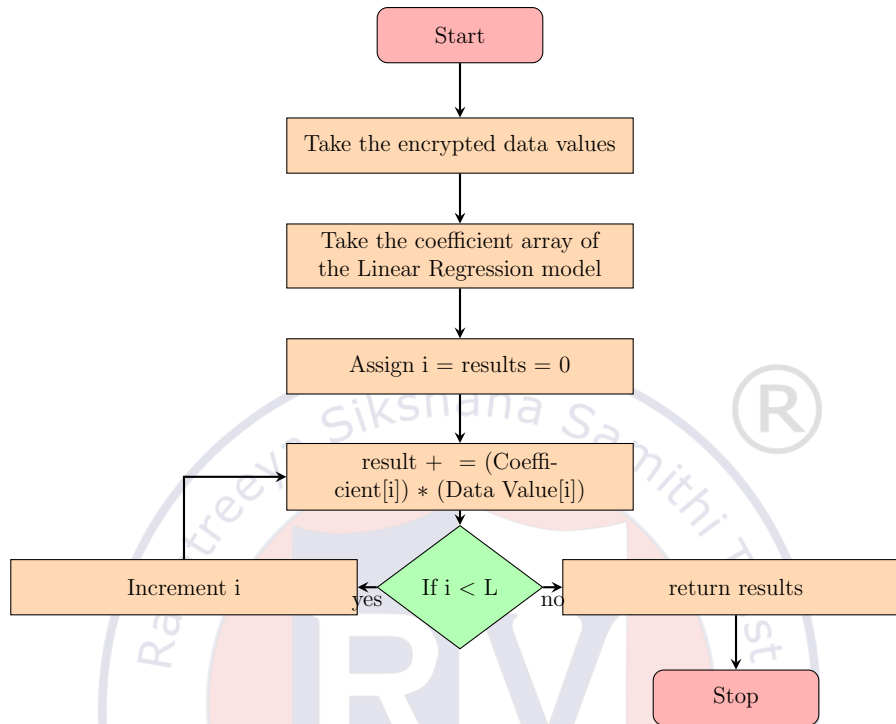


Figure 4.6: Compute the Answer

Figure 4.6 is the program responsible for performing the Linear Regression Analysis on the encrypted data and return the results. A Linear Regression analysis is aimed at finding the best fit line which is represented as a weighted sum of slopes and variables. Hence, a simple weighted sum is performed by this function.

4.3.8 Serialising Data

This function is similar to the previous serialisation function except here the encrypted answer is being serialised and since it is a single values, the process here is simpler.

4.3.9 Getting the Encrypted Answer

Once the answer has been dumped on the JSON file by the previous function, the answer needs to be read from the JSON file by the DO which is what is being done in this function.

4.3.10 Diffie-Hellman-Merkel Key Exchange Algorithm

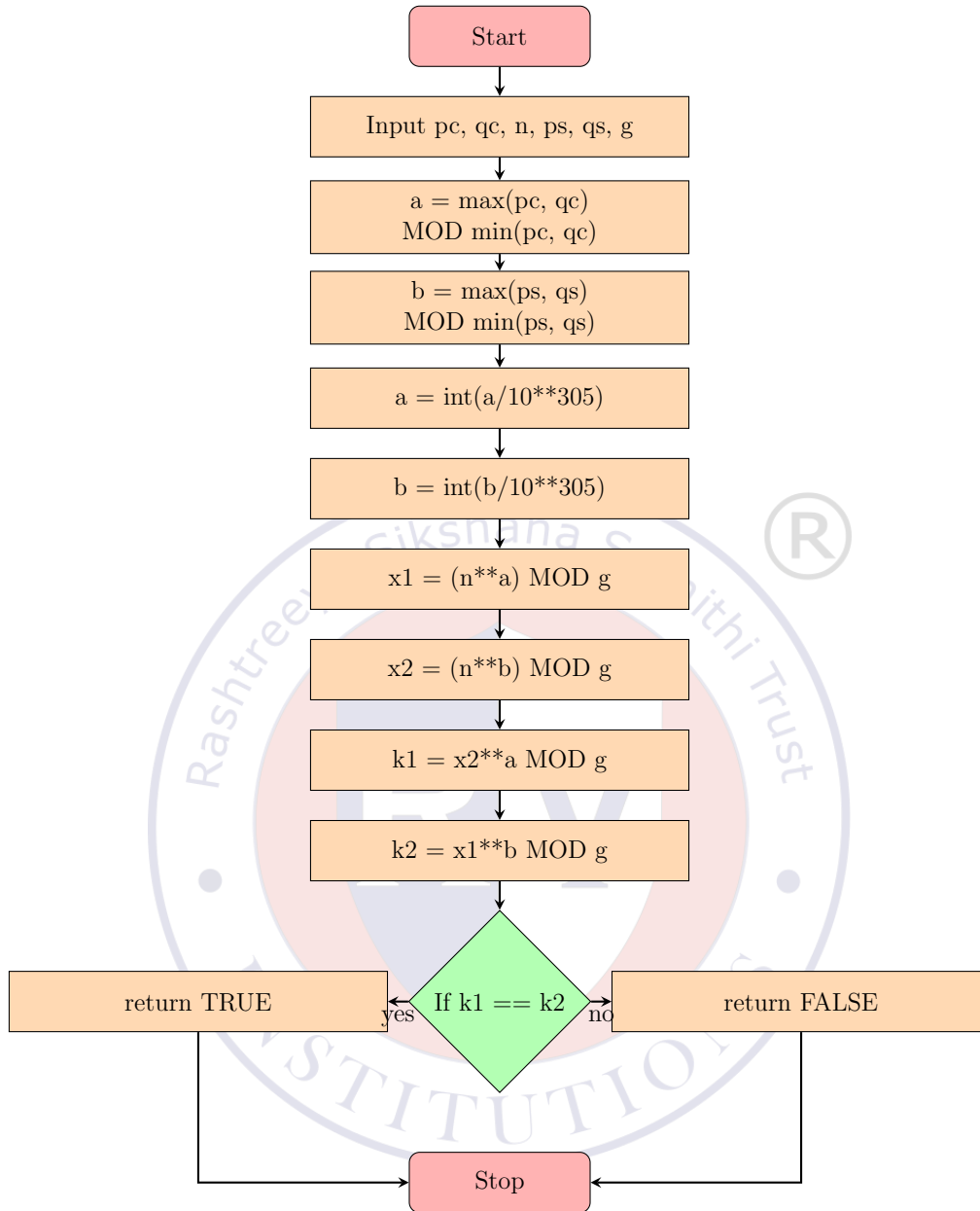


Figure 4.7: Diffie-Hellman-Merkel Key Exchange Algorithm

This algorithm is necessary to maintain communication integrity between entities. This is achieved by creating intermediate and final keys using public and private keys. If the final keys match, then the communication is secure as depicted in Figure 4.7.

This marks the end of the various functions that will be used throughout the framework. Now, the next part is writing the Prediction flow of the Code using the functions defined above.

4.4 Prediction using Privacy Preserving Framework

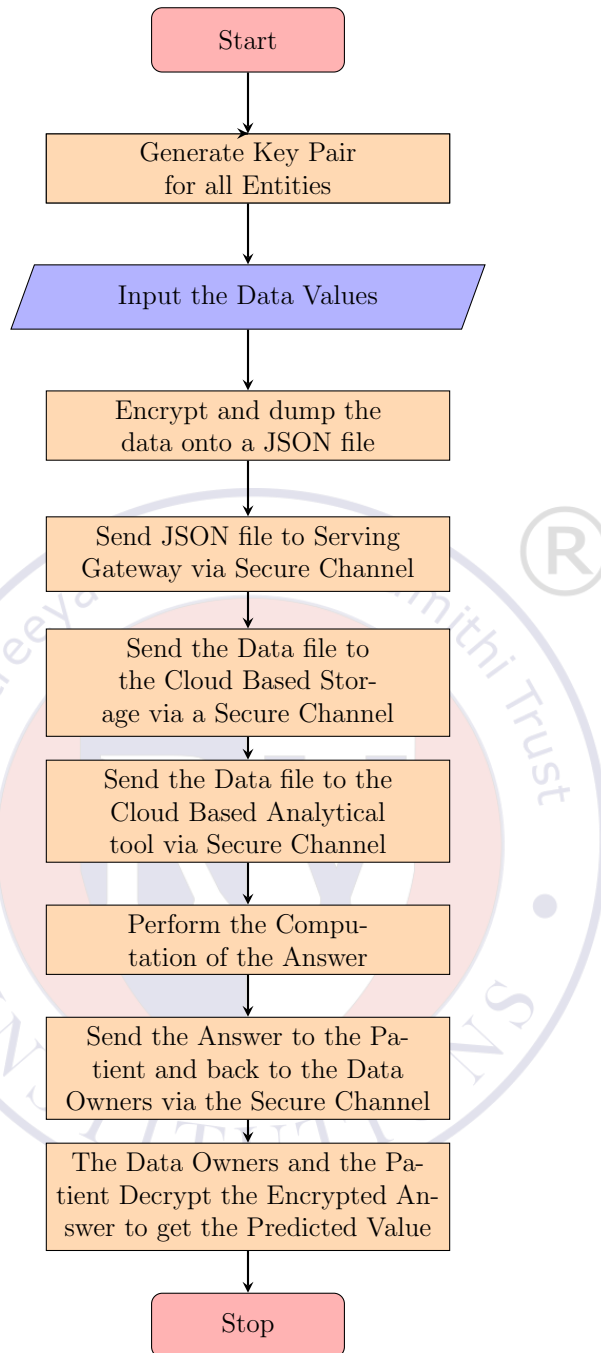


Figure 4.8: Prediction Framework

Once all the functions have been defined and coded, the framework as represented in Figure 4.8 can now perform the prediction operation on the incoming data. The flow of data between the entities is realised using the code below -

4.5 Training the ML Model

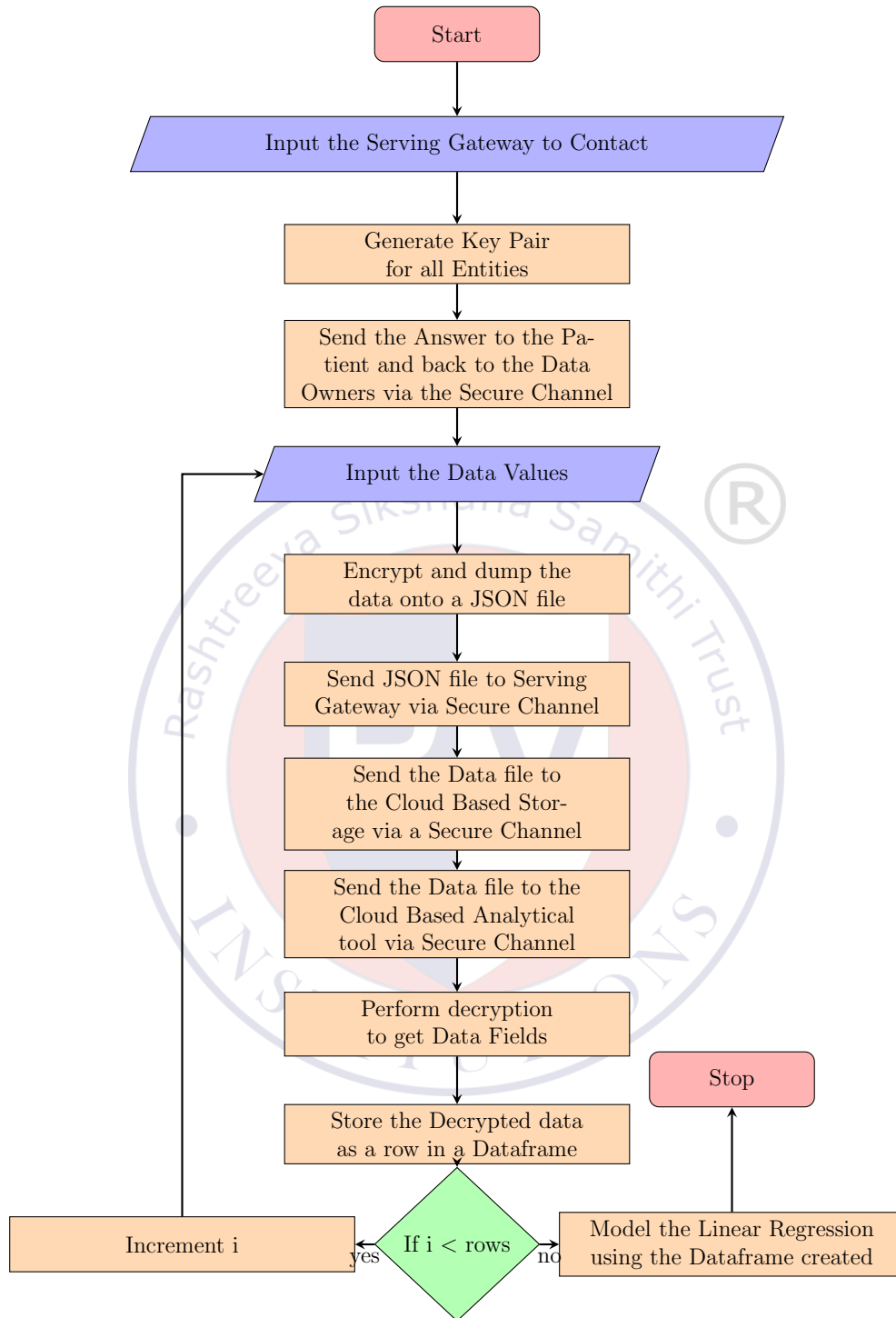


Figure 4.9: Training Framework

The above section discussed the flow of the Prediction Operation in the Privacy Preserving Framework. However, the Prediction process presumed that the model was trained on previous data. To remove this assumption, one need to code the process for training the Third Party Cloud Based Analytical Model.

To train the model, the past data is encrypted and sent to the Third Party Tool. In the proposed framework, the training of the model on encrypted data was not achieved. On the other hand, the Third Party and the DO come to a mutual agreement which allows the Third Party tool to decrypt the data and then use it for training (see Figure 4.9)

With the code run and results obtained, these must be quantified into useful comparative metrics that serve to highlight the accuracy and effectiveness of the trained model.





Chapter 5

Results & Discussions

CHAPTER 5

RESULTS & DISCUSSIONS

The Prediction and Training Framework implemented in chapter 4 was tested with various data inputs. The proposed work uses a basic CSV data set that tabulates the Medical Insurance Charges for people based on the factors like -

1. Age
2. Sex
3. BMI
4. Number of Children or Dependent Family Members
5. Whether the person is a smoker or not
6. The region of stay

After pre-processing the data, it is analysed using Linear Regression. There were three scenarios that were implemented -

- Regular Linear Regression Implementation
- Encrypted Framework Prediction
- Encrypted Framework Training

5.1 Execution Results

The execution begins with Framework Definition. This step involves defining the various entities in the framework and then generating key pairs for them. During entity definition, the user also has the liberty of choosing how many Serving Gateways are required in the framework. Once this step is done, the DO are asked to provide the data to be analysed. This data is a row data from the CSV file. This data is then encrypted and the DO choose which Serving Gateway to communicate with.

After this, the data is sent to the Cloud Based Services where the encrypted answer is formed. This encrypted answer is then sent to not only to the patient but back to the DO as well for decryption. The result for a single row of data is shown in Figure 5.1 -

```
CREATING THE FRAMEWORK
Enter the number of Serving Gateways present in the system : 4
DATA OWNERS
Enter the input data to be analysed32 1 28.880 0 0 1
ENCRYPTING THE DATA
Enter the name of the file that you want to store the data in : data
SERVING GATEWAY
Enter the Serving Gateway Number you want to communicate with : 6
Invalid Serving Gateway Number. Enter a valid value : 3
Sending Data to Selected Serving Gateway...
CLOUD BASED PLATFORM
Sending Data to Cloud Storage...
Sending Data to Insurance Company's Cloud Based Analysis Tool
Analysis Completed
Enter the name of the file that you want to store the answer in : answer
Sending Encrypted Answer to the Patient
PATIENT
Decrypting the Answer
5537.640897203209
Sending Encrypted Answer from Insurance Company to the Cloud Storage
Sending Encrypted Answer from Cloud Storage to Serving Gateway
Sending Encrypted Answer from Serving Gateway to Data Owners
DATA OWNER
Decrypting the Answer
5537.640897203209
```

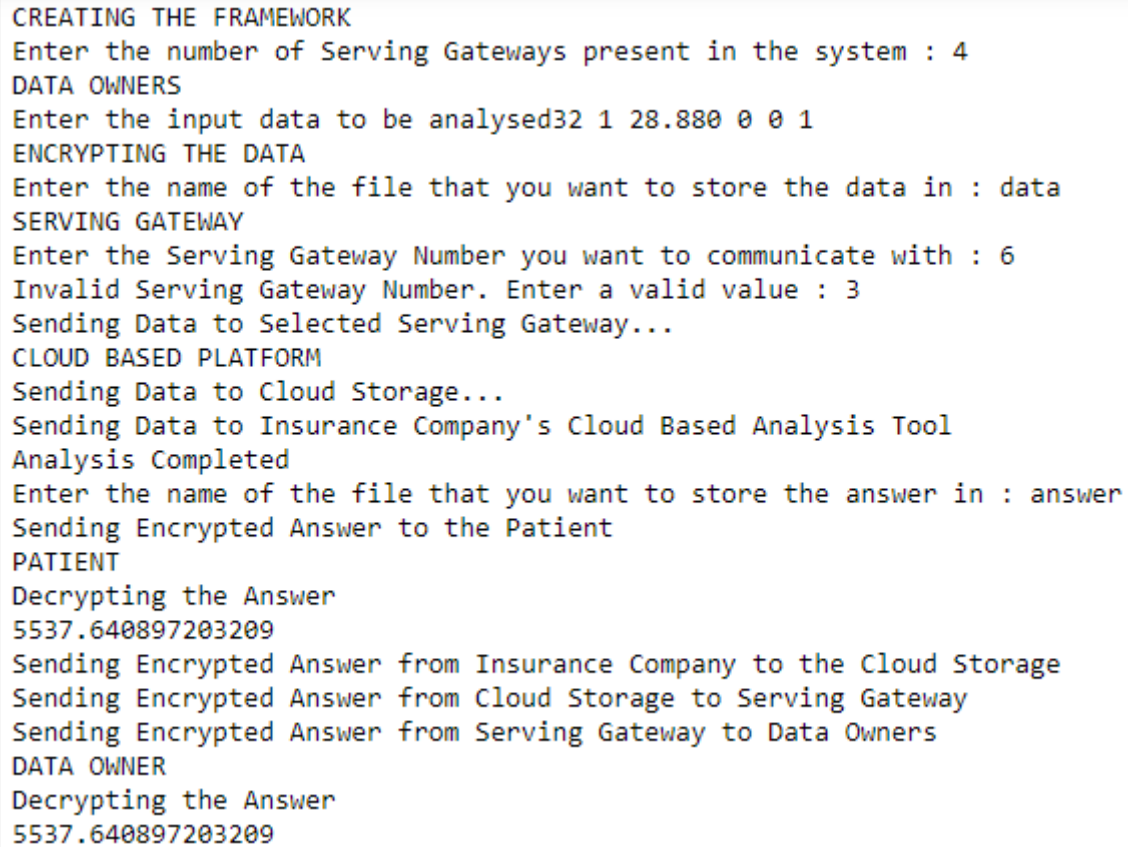


Figure 5.1: Output for Prediction Framework

On the other hand, when the Training Framework was executed, the framework had already been established and therefore, there was no need to create and define the entities again. It was also assumed that the Third Party Tools had an agreement with the DO that allowed the former to use the respective decrypting algorithms to decrypt the data for training purposes.

The DO sends the encrypted version of the data to the Third Party Tool via the Serving Gateway. The Cloud Based Storage then decrypts this data and then stores them in a Data Frame format. This data frame is then used to train the model.

5.2 Comparative Study

As mentioned in section 5.1, two parts of the framework were implemented - The Prediction Part and the Training Part. Both these frameworks seem to be functioning correctly and protects the data from being man-handled by malicious parties. However, a trade-off exists between the security provided by this framework and the time or memory it takes for execution. To quantify the said trade-off, a comparative study was performed

between four processes -

1. The Prediction Framework **vs** A regular Linear Regression Prediction process
2. The Training Framework **vs** A regular Linear Regression Training Process

The four processes were analysed by noting the time taken for the execution process. The results have been tabulated in Table 5.1.

Table 5.1: Timing Analysis of Predictive Framework Functions

Sub-process name	Time taken (s)
Creating the Linear Regression Model	0.0608
Creating and Storing Key Pairs	9.3406
Encrypting Single Row of Data	0.7909
Diffie Hellmann Key Exchange	0.0638
Result Computation	0.0927
Answer Decryption	0.0319

5.2.1 Predictive Framework vs Regular Linear Regression Prediction

To have a comparative study between the Regular Linear Regression Prediction and the framework that was developed, there is a need for finding the time required to perform the Regular Linear Regression Model. This has been tabulated in Table 5.2.

The regular Linear Regression model takes around **33.94ms** to execute. On the other hand, using the framework and predicting the value for a single row of data uses up **10.38 seconds**. This is a large difference between the two process with the framework taking more than 300 times the time required. This is the trade-off mentioned in section 5.2.

5.2.2 Training Framework vs Regular Linear Regression Training

The training part of the framework involved encrypting single lines of data and then sending it to the Third Party tool which decrypts the data and then uses it to train

Table 5.2: Timing Analysis of Training Framework Functions

Sub-process name	Time taken (s)
Data Pre-processing	0.0847
Creating and Storing Key Pairs	9.3406
Encrypting Single Row of Data	0.7909
Diffie Helmann Key Exchange	0.0638
Answer Decryption	0.0319
Creating the Linear Regression Model	2590

the model. This is a processor intensive task and takes **43.1 minutes** to execute. The process is quite time consuming and is a price one must pay to have data privacy.

The coding of the framework and hosting it on a local device opened the framework up for extensive testing. For the current scope, it was clear that the framework was to be tested against medical data. The results that were achieved matched the regular machine learning outputs with good accuracy. The only drawback was the time taken to create a Machine Learning model and training it. The extensive time and processor load is a trade-off that exists for using secure data handling constructs. The resulting outputs can be easily quantified and analysed so as to draw suitable conclusions as well mention the future scope of this work.



Chapter 6

Conclusion and Future Scope

CHAPTER 6

CONCLUSION AND FUTURE SCOPE

6.1 Conclusion

Over the course of the proposed work, it can be concluded that ensuring data privacy and security has become a prime concern for corporations and individuals across the globe. From the model implementation and analysis, one can infer that the use of HE and its sub-ordinates can help achieve the goal of maximum data security and privacy.

For the scope of the proposed work, a framework with five entities was proposed and then it was suitably designed. The functioning of the framework can be split into two frameworks - one for Prediction and one for Training. The Prediction Framework was coded and designed by assuming an analytical model being present with the Cloud Based Service Provider beforehand. The framework dealt with encryption of data, analysis on the data and then decryption by the DO. On the other hand, the Training Framework focused mainly on creating and training the analytical model hosted on the Cloud Based Platform. This model was trained by sending each row of data as a separately encrypted entity.

The framework was tested by using a CSV Medical Data consisting of 1338 rows and 7 fields. The Prediction Framework was able to achieve an error rate of 3.71×10^{-11} % making it highly accurate. The training framework was also tested by providing the past values from the data set and the coefficients of the analytical model were equal to the model that was trained under normal conditions. However, the overall framework took to execute making it highly processor intensive and time consuming. In the current state of the proposed framework, there exists a trade-off between the high accuracy and security provided and the execution time making the current model not desirable for high speed practical applications.

6.2 Future Scope

This scope of the current work is not that extensive and there is a lot of scope for future expansion -

- Optimizing the proposed framework to get the results in lesser time and without

overloading the processor

- Extend the framework by enabling analysis of unstructured data
- Implement a diverse range of analytical algorithms

6.3 Learning Outcomes of the Project

The learning outcomes of this project are -

- The importance of Data Privacy and Security as a concept and the Laws that enforce such concepts
- The Mathematical, Quantitative and Qualitative Analysis of Homomorphic Encryption and Partial Homomorphic Encryption
- The Mathematical, Quantitative and Qualitative Analysis of Machine Learning Tools like Linear Regression
- The functionalities of various Python and Sklearn Modules and Constructs
- The working of the functions involved with the Paillier PHE Library installed in Python
- The handling of JSON files and Data Structures

BIBLIOGRAPHY

- [1] A. H. Seh, M. Zarour, M. Alenezi, and A. K. e. a. Sarkar, "Healthcare data breaches: Insights and implications," *Healthcare (Basel)*, vol. 8(2), no. 133, 2020. DOI: <https://doi.org/10.3390/healthcare8020133>.
- [2] H. Journal, *December 2019 healthcare data breach report*, Dec. 2019. [Online]. Available: <https://www.hipaajournal.com/december-2019-healthcare-data-breach-report/>, (accessed: 03.01.2021).
- [3] H. Gharagyozyan. (). "A practical application of machine learning in medicine," [Online]. Available: <https://www.macadamian.com/learn/a-practical-application-of-machine-learning-in-medicine/>. (accessed: 01.01.2021).
- [4] M. Myszczyńska, P. Ojamies, and A. e. a. Lacoste, "Application of machine learning to diagnosis and treatment of neurodegenerative diseases," *Nature Reviews Neurology*, vol. 16, pp. 440–4556, 2020. DOI: <https://doi.org/10.1038/s41582-020-0377-8>.
- [5] Y. AONO, T. HAYASHI, L. T. PHONG, and L. WANG, "Privacy-preserving logistic regression with distributed data sources via homomorphic encryption," *IEICE Transactions on Information and Systems*, vol. E99.D, no. 8, pp. 2079–2089, 2016. DOI: 10.1587/transinf.2015INP0020.
- [6] H. Chen, R. Gilad-Bachrach, K. Han, Z. Huang, A. Jalali, K. Laine, and K. Lauter, "Logistic regression over encrypted data from fully homomorphic encryption," *BMC Medical Genomics*, vol. 11, no. 4, Oct. 2018. [Online]. Available: <https://www.microsoft.com/en-us/research/publication/logistic-regression-over-encrypted-data-from-fully-homomorphic-encryption-2/>.
- [7] V. N. Boddeti, *Secure face matching using fully homomorphic encryption*, 2018. arXiv: 1805.00577 [cs.CV].
- [8] A. Vengadapurvaja, G. Nisha, R. Aarth, and N. Sasikaladevi, "An efficient homomorphic medical image encryption algorithm for cloud storage security," *Procedia Computer Science*, vol. 115, pp. 643–650, 2017, 7th International Conference on Advances in Computing Communications, ICACC-2017, 22-24 August 2017, Cochin, India, ISSN: 1877-0509. DOI: <https://doi.org/10.1016/j.procs.2017.09.150>.

- [Online]. Available: <http://www.sciencedirect.com/science/article/pii/S1877050917319798>.
- [9] R. Rivest, L. Adleman, and M. Dertouzos, "On data banks and privacy homomorphisms," in *Foundations on Secure Computation*, Academia Press, 1978, pp. 169–179.
- [10] J. Andress, "Chapter 4 - auditing and accountability," in *The Basics of Information Security (Second Edition)*, J. Andress, Ed., Second Edition, Boston: Syngress, 2014, pp. 57–68, ISBN: 978-0-12-800744-0. DOI: <https://doi.org/10.1016/B978-0-12-800744-0.00004-X>. [Online]. Available: <http://www.sciencedirect.com/science/article/pii/B978012800744000004X>.
- [11] —, "Chapter 5 - cryptography," in *The Basics of Information Security (Second Edition)*, J. Andress, Ed., Second Edition, Boston: Syngress, 2014, pp. 69–88, ISBN: 978-0-12-800744-0. DOI: <https://doi.org/10.1016/B978-0-12-800744-0.00005-1>. [Online]. Available: <http://www.sciencedirect.com/science/article/pii/B9780128007440000051>.
- [12] W. Stallings, *Cryptography and Network Security: Principles and Practice*. Prentice Hall, 1999, ISBN: 9780138690175. [Online]. Available: <https://books.google.co.in/books?id=Dam9zrViJjEC>.
- [13] J. Gilchrist, "Encryption," in *Encyclopedia of Information Systems*, H. Bidgoli, Ed., New York: Elsevier, 2003, pp. 87–100, ISBN: 978-0-12-227240-0. DOI: <https://doi.org/10.1016/B0-12-227240-4/00054-X>. [Online]. Available: <http://www.sciencedirect.com/science/article/pii/B012227240400054X>.
- [14] A. C. Yao, "Protocols for secure computations," in *Proceedings of the 23rd Annual IEEE Symposium on Foundations of Computer Science*, Washington, DC, USA: IEEE Computer Society, 1982, pp. 160–164.
- [15] K. E. Lauter, M. Naehrig, and V. Vaikuntanathan, "Can homomorphic encryption be practical?" *IACR Cryptol. ePrint Arch.*, vol. 2011, p. 405, 2011.
- [16] P. Thaine, *Homomorphic encryption for beginners: A practical guide (part 2)*, As part of the Privacy-Preserving Natural Language Processing, 2019. [Online]. Available: <https://medium.com/privacy-preserving-natural-language-processing/>

homomorphic-encryption-for-beginners-a-practical-guide-part-2-the-fourier-transform-77bc9a1756, (accessed: 04.01.2021).

- [17] T. Graepel, K. Lauter, and M. Naehrig, “ML confidential: Machine learning on encrypted data,” in *Information Security and Cryptology – ICISC 2012*, T. Kwon, M.-K. Lee, and D. Kwon, Eds., Berlin, Heidelberg: Springer Berlin Heidelberg, 2013, pp. 1–21, ISBN: 978-3-642-37682-5.
- [18] I. Giacomelli, S. Jha, M. Joye, C. D. Page, and K. Yoon, *Privacy-preserving ridge regression with only linearly-homomorphic encryption*, Cryptology ePrint Archive, Report 2017/979, <https://eprint.iacr.org/2017/979>, 2017.
- [19] T. Ryffel, E. Dufour-Sans, R. Gay, F. Bach, and D. Pointcheval, *Partially encrypted machine learning using functional encryption*, 2019. arXiv: 1905.10214 [cs.LG].
- [20] J. W. Bos, K. E. Lauter, and M. Naehrig, “Private predictive analysis on encrypted medical data,” *Journal of biomedical informatics*, vol. 50, pp. 234–430, 2014.
- [21] A. Akavia, H. Shaul, M. Weiss, and Z. Yakhini, “Linear-regression on packed encrypted data in the two-server model,” Nov. 2019, pp. 21–32, ISBN: 978-1-4503-6829-2. DOI: 10.1145/3338469.3358942.
- [22] J. Zhan, S. Matawin, and C. L.W, “Privacy-preserving collaborative association rule mining,” in *19th Annual IFIP WG 11.3 Working Conference on Data and Applications Security*, S. Jadojia and D. Wijesekera, Eds., ser. Lecture Notes in Computer Science, vol. 3654, Storrs, CT, USA: Springer, Aug. 2005, pp. 153–165.